

Overview of Implemented Features in the UI

The deployed application is an interactive interface developed for **Lab Assignment 04: Image Denoising Using Autoencoders**. The application provides a simple way to test the trained autoencoder models, compare their performance, perform sensitivity experiments, and access the complete lab report. The interface follows a Maroon and White glassmorphism design and is divided into four main sections.

1 Tab 1: Live Denoising Studio

This section covers the deployment requirements specified in Section 11 of the lab syllabus. It allows users to provide an input image, add Gaussian noise, select an autoencoder model, and observe the denoising results.

1.1 Input Options

The application provides three ways to provide an input image.

MNIST Test Gallery:

Ten sample test images representing the digits 0 through 9 are included in the application. These images are taken directly from the evaluation test set used during the Colab experiments, allowing users to test the models without uploading an image.

Interactive Drawing Canvas:

Users can draw a digit on a 28×28 canvas. The canvas provides controls for changing the brush size, erasing parts of the drawing, clearing the entire canvas, and inverting the image polarity.

Custom Image Upload:

Users can upload PNG or JPG images using either drag-and-drop or the file browser. The uploaded image is automatically:

- Resized to 28×28 pixels.
- Converted to single-channel grayscale.
- Normalized to the range $[0, 1]$.
- Checked for background polarity and automatically inverted when required.

This ensures that the uploaded image follows the same basic format as the MNIST input data, where the digit appears as a white foreground on a black background.

1.2 Gaussian Noise Controls

The application provides controls for adding Gaussian noise to the input image.

- **Noise Level Slider:** The standard deviation σ can be adjusted continuously from 0.00 to 0.60.
- **Preset Values:** Quick selection buttons are available for $\sigma = 0.1, 0.2, 0.3, 0.4, 0.5$, matching the values used during the experiments.
- **Noise Toggle:** Users can enable or disable noise injection.
- **Re-seeding:** A re-roll option generates a new random noise pattern while keeping the selected noise level unchanged.

The noisy image can be generated using the Gaussian noise model:

$$I_{\text{noisy}} = I_{\text{clean}} + \epsilon \quad (1)$$

where

$$\epsilon \sim \mathcal{N}(0, \sigma^2) \quad (2)$$

and σ represents the standard deviation of the added Gaussian noise.

1.3 Autoencoder Model Selection

The application provides access to all five autoencoder architectures used in the experiment.

Denoising Autoencoder (DAE):

The model achieved a PSNR of 20.18 dB and an SSIM of 0.8677 in the final evaluation.

Undercomplete Autoencoder:

This model provides a compression ratio of approximately $24.5\times$ while achieving a PSNR of 19.69 dB.

Contractive Autoencoder (CAE):

The model focuses on learning stable latent representations and produced a PSNR of 15.37 dB. It also showed the lowest latent-space shift among the evaluated models.

Sparse Autoencoder (SAE):

This model uses an L_1 activity regularization term to encourage sparse latent representations. It achieved a PSNR of 11.86 dB.

Convolutional Autoencoder (Conv-AE):

The convolutional model has a relatively small parameter count of approximately 46.8k and achieved a PSNR of 9.73 dB.

The interface also provides a **5-model comparison mode**. When enabled, the same noisy input is passed through all five models, and their reconstructed outputs are displayed together for direct comparison.

1.4 Image Comparison and Visualization

The denoising results can be viewed using multiple visualization options.

Three-Panel View:

The clean image, noisy input, and reconstructed image are displayed side by side. Nearest-neighbor scaling is used with an $8\times$ enlargement so that individual pixels remain clearly visible.

Before-and-After Curtain:

An interactive slider allows users to move between the original and reconstructed images, making it easier to identify changes in individual strokes.

Residual Error Heatmap:

The application calculates the absolute pixel-wise reconstruction error using:

$$E(x, y) = |I_{\text{clean}}(x, y) - I_{\text{reconstructed}}(x, y)| \quad (3)$$

The resulting error map is displayed as a heatmap, helping identify areas where the model introduces blurring, removes important strokes, or fails to reconstruct parts of the digit accurately.

1.5 Real-Time Evaluation Metrics

The application calculates four primary image-quality metrics for the currently selected image.

Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2 \quad (4)$$

Lower values indicate smaller reconstruction errors.

Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i| \quad (5)$$

Lower values indicate better reconstruction accuracy.

Peak Signal-to-Noise Ratio (PSNR):

$$\text{PSNR} = 10 \log_{10} \left(\frac{\text{MAX}_I^2}{\text{MSE}} \right) \quad (6)$$

Higher PSNR values generally indicate better reconstruction quality.

Structural Similarity Index (SSIM):

SSIM measures the structural similarity between the clean and reconstructed images. Higher values indicate greater structural similarity.

The interface also calculates the PSNR gain over the noisy input:

$$\Delta\text{PSNR} = \text{PSNR}_{\text{reconstructed}} - \text{PSNR}_{\text{noisy}} \quad (7)$$

This shows how much the selected model improves the image compared with the original noisy input.

1.6 Export Options

The application provides two main export options.

Download Denoised Image:

The reconstructed image can be saved as a standalone PNG file.

Export Three-Panel Comparison:

The application can generate a combined comparison image containing the clean, noisy, and reconstructed images, along with relevant information such as the selected noise level and evaluation metrics.

2 Tab 2: Model Benchmarks Hub

This section presents the experimental comparison of the five autoencoder models and covers the comparative analysis required in Step 9 of the lab assignment.

2.1 Model Comparison Table

The table summarizes the main experimental results for each model, including:

- Number of parameters.
- Training time.
- Inference latency for 10,000 samples.
- MSE.
- MAE.
- PSNR.
- SSIM.
- Qualitative observations.

The models range from approximately 46.8k to 1.08M parameters, while their training times range from approximately 26.6 s to 80.6 s.

2.2 Experimental Plots

The section also includes the main plots generated from `finaldraft.ipynb`.

- **Figure 04:** Training and validation loss curves over 25 epochs.
- **Figure 06A:** Comparative bar charts showing the different evaluation metrics across the five models.
- **Figure 06B:** Pareto trade-off plot showing reconstruction quality in relation to parameter count.
- **Figure 07:** Distribution of latent activation sparsity and latent perturbation shift, represented as $\|\Delta z\|$.

These visualizations provide a clearer comparison of the models in terms of reconstruction quality, computational requirements, and latent-space behavior.

3 Tab 3: Parametric Experiments

This section contains the sensitivity experiments performed by varying the noise level and latent-space dimension.

3.1 Step 10: Noise-Level Sensitivity Analysis

The application includes Figure 08, which evaluates the model at noise levels ranging from

$$\sigma = 0.1 \text{ to } 0.5 \tag{8}$$

The corresponding table records the change in reconstruction quality as the noise level increases. For the best-performing model, the PSNR decreases from approximately 20.87 dB to 18.29 dB across the tested noise levels.

This experiment shows how increasing the amount of input noise affects the model’s ability to reconstruct the original image.

3.2 Step 11: Latent-Dimension Sensitivity Analysis

The application includes Figure 09, which evaluates different latent dimensions:

$$d \in \{8, 16, 32, 64, 128\} \tag{9}$$

The results show the relationship between the size of the latent representation and the reconstruction quality.

The corresponding table reports compression ratios ranging from approximately $98\times$ to $6.1\times$. The results also indicate that increasing the latent dimension beyond $d = 32$ provides comparatively smaller improvements in reconstruction quality, suggesting a point of diminishing returns.

4 Tab 4: Academic Lab Report Dossier

The final section contains the complete academic lab report and covers the requirements specified in Section 12 of the assignment.

The report includes:

- Title, aim, problem statement, and objectives.
- Dataset description and exploratory data analysis.
- Sample MNIST digits and distribution plots from Figures 01 and 02.
- Data preprocessing steps.
- Mathematical formulation of Gaussian noise.
- Architecture details of all five autoencoder models.
- Mathematical formulation of the MSE loss.
- L_1 activity regularization used in the Sparse Autoencoder.
- Contractive Frobenius-norm Jacobian penalty used in the Contractive Autoencoder.
- Description of the convolutional layers used in the Convolutional Autoencoder.
- Answers to all 10 analytical and post-lab questions.
- Conclusions based on the experimental results.
- References used during the experiment.

The report is integrated into the application so that the experimental results, analysis, and conclusions can be accessed without leaving the interface.

5 Design and Technical Architecture

The application follows a consistent Maroon and White editorial design throughout the interface. The dark theme uses a deep wine background (#0d0205), frosted burgundy panels, ruby accents (#be123c), and white typography.

5.1 Theme Switching

A theme switcher is available in the top-right corner of the interface. Users can switch between two themes:

- **Imperial Velvet Maroon:** Dark theme with a deep maroon background.
- **Pristine Alabaster and Royal Maroon:** Light theme with a white and maroon color scheme.

5.2 Technical Setup

The application is designed to be lightweight and self-contained. It runs using a native **Node.js HTTP server** at:

`http://localhost:3000`

The core interface does not depend on external runtime services. Image-processing operations and mathematical evaluation metrics are handled locally, allowing the application to remain responsive and usable offline once it has been set up.

Overall, the application combines the experimental results from the notebook with an interactive deployment interface. It allows users to test individual models, compare their outputs, analyze the effect of different noise levels and latent dimensions, and review the complete academic report in one place.